

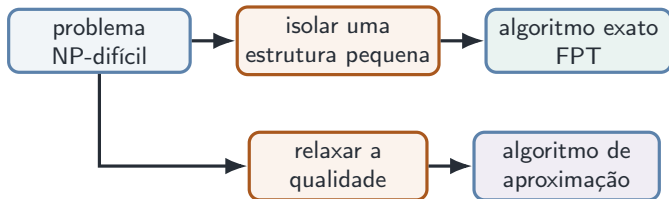
Aula 3: Algoritmos parametrizados e aproximação

Luís Felipe

UFF

20 de Agosto de 2026

Depois da NP-completude, ainda há escolhas algorítmicas



Duas perguntas centrais

1. **Onde está a explosão combinatória?** Podemos confiná-la a um parâmetro pequeno?
2. **Quanto de otimalidade podemos abrir mão?** Podemos provar uma garantia?

Uma instância parametrizada separa dados e estrutura

Problema parametrizado

Um problema parametrizado é um conjunto $Q \subseteq \Sigma^* \times \mathbb{N}$. Uma instância é (x, k) : x contém os dados, $n = |x|$ mede o tamanho total e k mede uma característica escolhida da instância.

classe	forma do tempo	significado
FPT	$f(k)n^c$, com c constante	a explosão fica confinada a k ; por exemplo, $2^k n^2$
XP	$n^{f(k)}$	para cada k fixo há um polinômio, mas seu grau pode depender de k ; e.g. n^k .

O mesmo problema clássico gera problemas parametrizados diferentes conforme a escolha de k .

Sobre o nome XP

Tempo exponencial considerando parâmetro fixo. XP é a notação tradicional da classe também chamada **slice-wise polynomial**: cada "fatia" formada pelas instâncias com k fixo é decidível em tempo polinomial.

E.g. Clique parametrizado pelo tamanho k ($\text{CLIQUE}(k)$). A enumeração de todos os conjuntos de k vértices leva tempo $O(n^k)$. Para k fixo, esse tempo é polinomial; portanto, $\text{CLIQUE}(k) \in \text{XP}$.

FPT e XP podem se separar rapidamente na prática

Considere $k = 10$ e ignore constantes menores.

tempo	$n = 10^3$	$n = 10^6$
$2^k n^2$	$\approx 10^9$	$\approx 10^{15}$
n^k	10^{30}	10^{60}

Notação O^*

Escrevemos $O^*(f(k))$ quando omitimos fatores polinomiais no tamanho da entrada:

$$O^*(2^k) = 2^k n^{O(1)}.$$

Mas FPT não significa automaticamente rápido

$2^k n$ pode ser útil; $2^{2^k} n$ provavelmente não. A dependência paramétrica e o fator polinomial importam.

Um bom parâmetro mede a parte que cria escolhas

Dois testes

Aplicação: há razão para k ser pequeno nas instâncias de interesse?

Algoritmo: a explosão combinatória pode depender somente de k ?

Indícios úteis:

- solução com poucos eventos;
- poucos conflitos a reparar;
- grafo próximo de uma classe simples;
- pequena largura de uma decomposição.

Exemplos

- **Vertex Cover:** k é o número de vértices escolhidos;
- **Feedback Vertex Set:** k vértices removem todos os ciclos;
- **treewidth t :** mede quão próximo o grafo está de uma árvore e viabiliza PD local;
- **Closest String:** d limita quantas posições cada entrada pode discordar do centro;
- rearranjos de genomas: d limita o número total ou máximo de operações.

Em bioinformática, há vários candidatos naturais

parâmetro	interpretação	risco
valor da solução d	número de diferenças, movimentos ou erros permitidos	pode crescer com sequências divergentes
número de entradas m	número de genomas, sequências ou árvores	fixá-lo pode não remover a dificuldade
tamanho do padrão ℓ	comprimento de padrão, primer ou janela	pode ser grande por definição experimental
alfabeto $ \Sigma $	DNA tem alfabeto pequeno	muitas intratabilidades permanecem até no caso binário
largura estrutural	treewidth, número de reticulações, discordâncias	“precisa” refletir a estrutura real dos dados

Podemos estudar diferentes parâmetros para um problema. Cada um responde uma hipótese e e pode gerar resultados diferentes. Não existe “o parâmetro” de um problema.

3-Coloração ilustra para-NP-dificuldade

Para-NP-dificuldade

Um problema parametrizado é **para-NP-difícil** se permanece NP-difícil para algum valor constante do parâmetro.

Exemplo: Coloração parametrizada pelo número q de cores

A pergunta é se G admite uma coloração própria com no máximo q cores. Para $q = 3$, obtemos exatamente **3-Coloração**, que é NP-completa (Aula 16, APA/ASA). Logo, essa parametrização é para-NP-difícil.

Por que isso exclui até XP?

Se existisse um algoritmo XP $n^{f(q)}$, a fatia $q = 3$ seria resolvida em $n^{f(3)}$, um tempo polinomial. Isso resolveria 3-Coloração em tempo polinomial. Portanto, **se $P \neq NP$** , o problema não pertence a XP e, em particular, não é FPT.

Para-NP-dificuldade é uma barreira mais forte que W[1]-dificuldade.

Parâmetros combinados descrevem mais de uma restrição

Parametrização multivariada

O parâmetro pode ser uma tupla, por exemplo (d, m) , e o tempo pode ser

$$f(d, m) n^{O(1)}.$$

Exemplo em grafos: q -Coloração em treewidth t

Dada uma decomposição em árvore, uma PD guarda a cor dos vértices presentes em cada bolsa. Uma bolsa possui no máximo $t + 1$ vértices, portanto há no máximo

$$q^{t+1}$$

atribuições locais. O tempo é $q^{O(t)} n^{O(1)}$: FPT no parâmetro combinado (q, t) .

- q limita o domínio de escolhas em cada vértice;
- t limita quantos vértices precisam ser lembrados simultaneamente;
- em bioinformática, combinações análogas usam distância d , número de entradas m e larguras estruturais.

Paradigmas FPT procuram uma fronteira pequena

paradigma	ideia	exemplo
kernelização	remover redundância por regras seguras até o tamanho depender apenas de k	Vertex Cover
árvore de busca	encontrar um conflito e ramificar apenas sobre reparos que cobrem todas as soluções	Vertex Cover; Closest String
PD estrutural	guardar estados somente sobre uma fronteira de largura limitada	coloração em treewidth t
compressão iterativa	inserir dados aos poucos e reduzir uma solução de tamanho $k + 1$ para k	Odd Cycle Transversal

A prova é o ponto central

Em cada técnica precisamos mostrar por que a informação descartada é irrelevante, por que algum ramo preserva uma solução ou por que o estado local resume todo o passado.

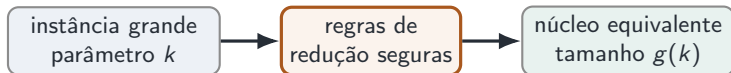
Kernelização formaliza pré-processamento com garantia

Kernelização

Um algoritmo de kernelização transforma, em tempo polinomial, (I, k) em uma instância equivalente (I', k') tal que

$$|I'| + k' \leq g(k).$$

Se g é polinomial, temos um **kernel polinomial**.



Obs.:

Todo problema decidível é FPT se, e somente se, possui algum kernel computável. Ter **kernel polinomial** é uma propriedade adicional e mais forte.

Uma regra de redução precisa preservar a resposta

Regra segura

Uma transformação $(I, k) \mapsto (I', k')$ é segura quando

$$(I, k) \text{ é sim} \iff (I', k') \text{ é sim.}$$

Não basta

- “parece irrelevante”;
- “raramente aparece”;
- “o solver ficou mais rápido”.

Isso pode ser uma boa heurística, mas não prova equivalência.

É preciso provar

- como uma solução antiga gera uma nova;
- como uma solução nova é levantada para a instância antiga;
- por que o tamanho final depende apenas do parâmetro.

Árvores de busca ramificam sobre reparos inevitáveis



Se cada passo reduz o orçamento em pelo menos 1, a profundidade é no máximo k . Com no máximo $r(k)$ filhos por nó:

$$\text{número de nós} \leq r(k)^k.$$

O centro da prova

Não basta enumerar reparos plausíveis: precisamos provar que **algum ramo preserva toda solução possível**.

Closest String procura um centro sob distância de Hamming

Closest String – decisão

Dados m strings $x_1, \dots, x_m \in \Sigma^L$ e um inteiro d , existe uma string $y \in \Sigma^L$ tal que

$$d_H(y, x_i) \leq d \quad \text{para todo } i \in \{1, \dots, m\}?$$

Distância de Hamming

$d_H(a, b)$ é o número de posições em que duas strings de mesmo comprimento diferem.

A	C	G	T	A	C
A	T	G	T	T	C
	↑			↑	

$$d_H = 2$$

Vamos considerar o parâmetro a distância permitida d .

As strings podem ser vistas como uma matriz de colunas

x_1	A	C	G	T	A	C	G	A
x_2	A	C	T	T	A	C	G	T
x_3	A	C	G	T	G	C	G	A
x_4	A	C	G	T	A	C	A	A
	colunas boas		colunas ruins					

- uma coluna é **boa** se todas as strings têm o mesmo símbolo;
- é **ruim** se contém ao menos dois símbolos distintos.

Primeira observação

As escolhas combinatórias estão apenas nas colunas ruins.

Colunas boas podem ser fixadas e removidas

Regra CS.1

Se $x_1[j] = \dots = x_m[j] = a$, fixe $y[j] = a$ e remova a coluna j .

Por que a regra é segura?

Se uma solução usa $y[j] \neq a$, trocar apenas $y[j]$ por a reduz em 1 a distância para **todas** as strings. Logo existe uma solução ótima que concorda com a coluna boa.

Exemplo

Se todas as entradas começam por AC, então existe um centro ótimo que também começa por AC. Fixamos essas duas posições e resolvemos apenas os sufixos. Depois, recolocamos AC na resposta.

Ou seja, essa regra elimina informação redundante, **não é uma parte difícil** do problema.

Uma instância sim possui no máximo md colunas ruins

Lema

Se existe um centro y com $d_H(y, x_i) \leq d$ para todo i , então a matriz possui **no máximo md colunas ruins**.

Prova

Em cada coluna ruim j , pelo menos uma string $x_{i(j)}$ difere de y nessa posição.

Cobramos a coluna j à string $x_{i(j)}$.

Cada string pode receber no máximo d cobranças, pois difere de y em no máximo d posições. Com m strings, há no máximo md colunas ruins.

Regra CS.2

Se há mais de md colunas ruins, responda **não**. Caso contrário, após CS.1 resta comprimento $L' \leq md$.

Um conflito fornece apenas $d + 1$ posições relevantes

Mantenha uma candidata z , inicialmente $z = x_1$. Suponha que alguma string x_i viole a restrição, ou seja:

$$d_H(z, x_i) > d.$$

Escolha quaisquer $d + 1$ posições P nas quais z e x_i diferem.

Fato inevitável

Todo centro y deve satisfazer $y[p] = x_i[p]$ para pelo menos um $p \in P$.

Por quê?

Se $y[p] \neq x_i[p]$ em todas as $d + 1$ posições de P , então

$$d_H(y, x_i) \geq d + 1,$$

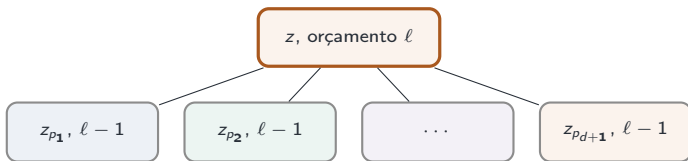
contradizendo que y é centro.

Como não sabemos qual posição é a correta; ramificamos nas $d + 1$ possibilidades.

A ramificação aproxima a candidata de alguma solução

Para cada $p \in P$, crie uma candidata z_p igual a z , exceto por

$$z_p[p] \leftarrow x_i[p].$$



Ramo correto

Se existe y com $d_H(y, z) \leq \ell$, o fato anterior garante um $p \in P$ com $y[p] = x_i[p]$. Como $z[p] \neq x_i[p]$, nesse ramo subtraímos uma unidade na distância:

$$d_H(y, z_p) = d_H(y, z) - 1 \leq \ell - 1.$$

O invariante prova a correção do algoritmo

Subproblema recursivo

Dados z e ℓ , decidir se existe um centro y com $d_H(y, z) \leq \ell$.

1. Se z já está a distância no máximo d de todas as entradas, devolva **sim**.
2. Se $\ell = 0$ e z não é centro, devolva **não**.
3. Escolha x_i com $d_H(z, x_i) > d$, forme P com $d + 1$ diferenças e ramifique.

Invariante

Se o nó atual contém alguma solução y dentro do orçamento ℓ , então pelo menos um filho contém a mesma solução dentro do orçamento $\ell - 1$.

Na raiz, $z = x_1$ e $\ell = d$. Qualquer centro satisfaz $d_H(y, x_1) \leq d$, portanto o invariante começa verdadeiro.

O tempo exponencial depende apenas de d

- a profundidade é no máximo d , pois ℓ diminui em cada nível;
- o fator de ramificação é no máximo $d + 1$;
- a árvore possui $O((d + 1)^d)$ nós;
- após o pré-processamento, o comprimento é no máximo md .

Tempo total

$$O(mL + md(d + 1)^d).$$

Logo, **Closest String é FPT parametrizado por d .**

Uma sutileza sobre kernelização

O comprimento reduzido md depende também de m . Portanto, essa redução sozinha não é um kernel em função apenas de d .

O parâmetro mostrou onde estava a dificuldade

1. **Remoção segura:** colunas unânimes podem ser fixadas e retiradas.
2. **Limite estrutural:** em uma instância sim, restam no máximo md colunas ruins.
3. **Testemunha de conflito:** uma string a distância maior que d fornece $d + 1$ posições; toda solução precisa concordar com ela em alguma dessas posições.
4. **Busca limitada:** ramificamos em no máximo $d + 1$ escolhas por nível e consumimos uma unidade do orçamento.

O algoritmo é exato e evita enumerar todas as $|\Sigma|^L$ strings possíveis.

Pergunta reutilizável

Quando uma candidata viola uma restrição, consigo extrair um conjunto pequeno que toda solução precisa “acertar”?

Uma decomposição em árvore organiza o grafo em bolsas

Uma **decomposição em árvore** de $G = (V, E)$ é um par

$$\mathcal{T} = (T, \{X_t\}_{t \in V(T)}),$$

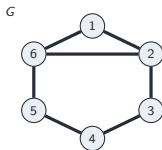
em que T é uma árvore e cada **bolsa** X_t é um subconjunto de V .

Ela deve satisfazer:

1. $\bigcup_{t \in V(T)} X_t = V$;
2. para toda aresta $uv \in E$, alguma bolsa contém u e v ;
3. para cada $v \in V$, os nós t com $v \in X_t$ induzem um subgrafo conexo de T .

O que a terceira condição impede?

Para cada vértice, as bolsas que o contêm devem formar um trecho conectado de T : ele não pode desaparecer da decomposição e reaparecer depois.



O vértice 6 aparece em todas as bolsas; o vértice 2, em duas bolsas consecutivas. Cada aresta de G aparece em alguma bolsa.

Treewidth mede o tamanho da maior fronteira necessária

Largura e treewidth

A largura de $\mathcal{T} = (T, \{X_t\})$ e a treewidth de G são

$$\text{width}(\mathcal{T}) = \max_{t \in V(T)} |X_t| - 1, \quad \text{tw}(G) = \min_{\mathcal{T}} \text{width}(\mathcal{T}).$$

Assim, $\text{tw}(G)$ é a menor largura entre todas as decomposições de G .

grafo	treewidth	interpretação
árvore com arestas	1	bolsas de tamanho 2 bastam
ciclo	2	alguma bolsa precisa ter tamanho 3
clique K_r	$r - 1$	seus r vértices aparecem juntos em alguma bolsa
grade $r \times r$	r	grafos esparsos também podem ter largura grande

Por que subtrair 1?

Assim, árvores têm largura 1 e grafos sem arestas têm largura 0. Treewidth pequena indica **separadores pequenos**, não poucas arestas.

Nós, bolsas e vértices do grafo são objetos distintos

Quem pertence a quê?

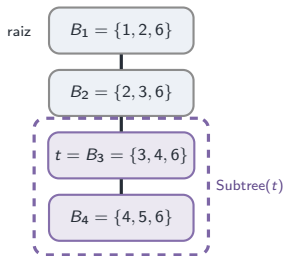
- $G = (V, E)$ é o **grafo original**; seus elementos são vértices de V ;
- T é a **árvore de decomposição**; $t, u \in V(T)$ são nós de T ;
- $X_t \subseteq V$ é a **bolsa** associada ao nó t ;
- após enraizar T , $\text{Subtree}(t)$ contém t e todos os seus descendentes em T .

Definimos

$$V_t = \bigcup_{u \in \text{Subtree}(t)} X_u.$$

V_t é o conjunto de **vértices de G** que aparecem em alguma bolsa da subárvore de t ; em geral, V_t não é uma bolsa.

u percorre **nós da árvore T** ; os elementos reunidos pelas bolsas X_u são vértices do grafo G .



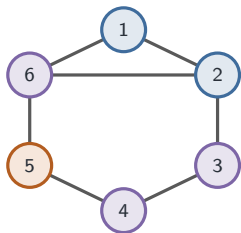
Para $t = B_3$:

$$\text{Subtree}(t) = \{B_3, B_4\},$$

$$X_t = \{3, 4, 6\},$$

$$V_t = X_{B_3} \cup X_{B_4} = \{3, 4, 5, 6\}.$$

A bolsa X_t separa o interior do exterior



- exterior: $V \setminus V_t = \{1, 2\}$
- fronteira: $X_t = \{3, 4, 6\}$
- interior: $V_t \setminus X_t = \{5\}$

No exemplo anterior,

$$V_t = \{3, 4, 5, 6\}, \quad X_t = \{3, 4, 6\}.$$

Logo,

$$V_t \setminus X_t = \{5\}, \quad V \setminus V_t = \{1, 2\}.$$

Propriedade de separação

Não existe aresta do grafo original G ligando diretamente o interior ao exterior:

$$E_G(V_t \setminus X_t, V \setminus V_t) = \emptyset.$$

Pode haver arestas do interior para X_t e de X_t para o exterior.

A expressão fala de arestas de G , não das arestas da árvore de decomposição T . Toda conexão entre o interior e o exterior precisa passar pela bolsa X_t .

Uma decomposição nice torna a PD local

Uma decomposição **nice** (normalizada) possui apenas os tipos de nó abaixo; podemos obtê-la sem aumentar a largura.

folha	$X_t = \emptyset$
introduz v	$X_t = X_c \cup \{v\}$
esquece v	$X_t = X_c \setminus \{v\}$
junção	$X_t = X_{c_1} = X_{c_2}$

c (ou c_1, c_2) denota o filho (ou os filhos) de t .

Por que as operações locais bastam?

Depois de processar a subárvore de t , toda interação com o futuro passa por X_t . Assim, a tabela precisa registrar apenas o comportamento da solução nessa bolsa.

Para **Independent Set**, guardaremos

$$S = I \cap X_t,$$

isto é, quais vértices da fronteira já pertencem ao conjunto independente.

Ideia da PD estrutural

Soluções parciais com a mesma escolha em X_t são indistinguíveis para o restante do grafo; conservamos apenas a de melhor valor.

Independent Set memoriza a interseção com a bolsa

Para um nó t e um subconjunto $S \subseteq X_t$, define

$$\text{DP}[t, S] = \max\{|I| : I \subseteq V_t, I \text{ é independente e } I \cap X_t = S\}.$$

Significado do estado

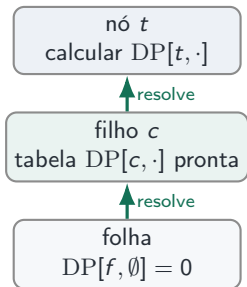
S registra exatamente quais vértices da fronteira pertencem à solução. Entre todas as soluções parciais compatíveis com S , conservamos apenas a de maior tamanho.

- se S não é independente, $\text{DP}[t, S] = -\infty$;
- como $|X_t| \leq \text{tw}(G) + 1$, há no máximo $2^{\text{tw}(G)+1}$ estados por bolsa;
- se a raiz r tem bolsa vazia, a resposta é $\text{DP}[r, \emptyset]$.

Invariante

O estado $I \cap X_t$ contém toda a informação do passado necessária para completar a solução.

A recursão sobe das folhas para a raiz



O que é recursivo?

Para calcular a tabela de um nó t , usamos somente as tabelas de seus filhos. Esses filhos são resolvidos da mesma maneira, até alcançar uma folha.

Para cada nó t :

1. resolvemos primeiro cada filho de t ;
2. para cada $S \subseteq X_t$, aplicamos a regra local do tipo de t ;
3. armazenamos $DP[t, S]$ e subimos para o pai.

Recursão na definição; preenchimento de baixo para cima

Podemos descrever o algoritmo recursivamente, mas cada tabela é calculada uma única vez. Isso é programação dinâmica, não enumeração repetida das mesmas subárvores.

Introduzir v acrescenta uma nova escolha

Considere um nó t com filho c e $X_t = X_c \cup \{v\}$.

Regra de introdução

Para cada conjunto independente $S \subseteq X_t$,

$$DP[t, S] = \begin{cases} DP[c, S], & v \notin S, \\ DP[c, S \setminus \{v\}] + 1, & v \in S. \end{cases}$$

Se $v \in S$, a entrada só é válida quando v não é adjacente a nenhum vértice de $S \setminus \{v\}$.

Exemplo local

Se

$$X_c = \{3, 6\}, \quad X_t = \{3, 4, 6\},$$

o vértice introduzido é $v = 4$.

Para $S = \{6\}$, o novo vértice fica fora:

$$DP[t, \{6\}] = DP[c, \{6\}].$$

Para $S = \{4, 6\}$, o novo vértice entra:

$$DP[t, \{4, 6\}] = DP[c, \{6\}] + 1.$$

Intuição

O filho já resolveu todo o interior anterior. O nó de introdução decide apenas se o novo vértice v pertence à solução.

Esquecer maximiza; juntar evita uma contagem dupla

Esquecimento de v

Se $X_t = X_c \setminus \{v\}$, o estado $S \subseteq X_t$ não informa mais se v foi escolhido.

Consultamos as duas possibilidades:

$$DP[t, S] = \max\{DP[c, S], \\ DP[c, S \cup \{v\}]\}.$$

Exemplo: ao esquecer 3 e manter $S = \{6\}$, comparamos os estados $\{6\}$ e $\{3, 6\}$ do filho e guardamos o melhor valor.

A mesma ideia nos dois casos

Estados diferentes do passado que produzem a mesma escolha na bolsa tornam-se indistinguíveis; conservamos somente o melhor valor.

Junção

Se $X_t = X_{c_1} = X_{c_2}$, os dois filhos representam interiores diferentes, mas ambos incluem os vértices escolhidos em S :

$$DP[t, S] = DP[c_1, S] + DP[c_2, S] - |S|.$$

Se os filhos têm valores 5 e 4 para o mesmo S com $|S| = 2$, então

$$DP[t, S] = 5 + 4 - 2 = 7.$$

Subtraímos $|S|$ porque a fronteira foi contada nos dois filhos.

A exponencial depende da treewidth

Para uma decomposição de largura t :

- cada bolsa possui no máximo $t + 1$ vértices;
- há no máximo 2^{t+1} subconjuntos por bolsa;
- uma decomposição nice tem $O(tn)$ nós e transições em $t^{O(1)}$.

Assim, dado G e uma decomposição de largura t , Maximum Independent Set é resolvido em

$$2^t t^{O(1)} n.$$

A decomposição precisa ser obtida

Há algoritmos FPT que, dados G e t , encontram uma decomposição de largura limitada ou concluem que $\text{tw}(G) > t$. Para cada t fixo, Bodlaender mostrou um algoritmo linear em n .

Técnica aplicável em outros problemas

Vertex Cover usa estados semelhantes; q -Coloração guarda uma cor por vértice da bolsa, produzindo q^{t+1} estados. Muitos problemas tornam-se FPT em t .

Uma equivalência clássica pode não preservar o parâmetro

Redução parametrizada $A \leq_{\text{fpt}} B$

Uma transformação leva (x, k) a (x', k') de modo que:

1. (x, k) é sim se, e somente se, (x', k') é sim;
2. $k' \leq g(k)$ para alguma função computável g ;
3. a transformação leva tempo $f(k)|x|^{O(1)}$.

Vertex Cover vs Independent Set

Para todo grafo $G = (V, E)$,

$$C \text{ é vertex cover} \iff V \setminus C \text{ é conjunto independente.}$$

Porém, um cover de tamanho k corresponde a um conjunto independente de tamanho $|V| - k$. O novo parâmetro depende de $|V|$, não apenas de k .

Consequência

A equivalência serve como redução clássica, mas não transfere FPT entre as parametrizações naturais. Vertex Cover(k) é FPT; Independent Set(k) é W[1]-completo.

Vertex Cover possui uma árvore de busca 2^k

Problema parametrizado

Dados $G = (V, E)$ e k , existe $C \subseteq V$, $|C| \leq k$, que contém ao menos uma extremidade de cada aresta?

Escolha uma aresta uv . Toda cobertura precisa conter u ou v .

1. ramo u : remova u e suas arestas; chame o algoritmo com $k - 1$;
2. ramo v : remova v e suas arestas; chame o algoritmo com $k - 1$;
3. aceite se algum ramo aceitar; rejeite se $k < 0$.

Correção e tempo

Os dois ramos cobrem todas as soluções possíveis. A profundidade é no máximo k e cada nó tem dois filhos:

$$T(k) \leq 2T(k - 1) + n^{O(1)} = O(2^k n^{O(1)}).$$

Vertex Cover também possui um kernel simples

Regra de alto grau

Se $\deg(v) > k$, então v pertence a toda cobertura com no máximo k vértices. Caso contrário, seria necessário escolher mais de k vizinhos de v .

Inclua v , remova-o com suas arestas e substitua k por $k - 1$. Remova também vértices isolados.

Após aplicar a regra exaustivamente, o grau máximo é no máximo k .

Limite do kernel

Se restam mais de k^2 arestas, a resposta é não: como o grau máximo é k , uma cobertura com k vértices cobre no máximo k^2 arestas.

Caso contrário, $|E| \leq k^2$. Como removemos os vértices isolados e cada aresta possui dois extremos,

$$|V| \leq 2|E| \leq 2k^2.$$

Resultado

Obtemos uma instância equivalente com $|E| \leq k^2$ e $|V| \leq 2k^2$. É um kernel polinomial; há kernels mais fortes para o problema.

A hierarquia W classifica a dificuldade parametrizada

$$\text{FPT} \subseteq W[1] \subseteq W[2] \subseteq \dots \subseteq \text{XP}.$$

Ideia da hierarquia

As classes $W[t]$ reúnem problemas parametrizados cuja estrutura combinatória é comparada, por reduções parametrizadas, à satisfatibilidade ponderada de certos circuitos booleanos.

Uma atribuição de **peso** k torna exatamente k variáveis verdadeiras. As restrições impostas aos circuitos determinam os diferentes níveis da hierarquia.

classe	problemas representativos
FPT	Vertex Cover parametrizado pelo tamanho da cobertura
$W[1]$	Clique e Independent Set parametrizados pelo tamanho da solução
$W[2]$	Dominating Set e Set Cover parametrizados pelo número de conjuntos

Consequência

Acredita-se que $\text{FPT} \neq W[1]$. Portanto, demonstrar que um problema é $W[1]$ -difícil é evidência de que ele não admite algoritmo

$$f(k) n^{O(1)}$$

para o parâmetro considerado. O problema ainda pode pertencer a XP, com tempo como $n^{f(k)}$.

$W[1]$ e $W[2]$ têm problemas canônicos diferentes

$W[1]$

Completo pelo tamanho k da solução:

- **Clique**;
- **Independent Set**.

Intuição: selecionar k objetos que precisam satisfazer muitas compatibilidades entre pares.

$W[2]$

Completo pelo tamanho k da solução:

- **Dominating Set**;
- **Set Cover**;
- **Hitting Set**.

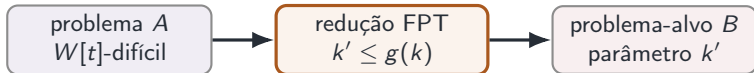
Intuição: selecionar k objetos que devem atingir ou cobrir todas as restrições.

Optimal Primer Cover – OPC

Na abstração geral de cobertura de alvos por primers, parametrizar apenas pelo número de primers selecionados herda a dificuldade de **Set Cover**.

Como provar W-dificuldade

Para mostrar que um problema B é $W[t]$ -difícil:



1. prove a equivalência das respostas;
2. prove que o novo parâmetro depende apenas do antigo;
3. prove tempo FPT da transformação.

Dificuldade versus completude

$W[t]$ -**difícil** exige apenas a redução a partir de um problema difícil.

$W[t]$ -**completo** exige também provar que o problema pertence a $W[t]$.

Set Cover reduz à cobertura abstrata de primers

Set Cover parametrizado por k

Entrada: universo $U = \{u_1, \dots, u_n\}$, família $\mathcal{S} = \{S_1, \dots, S_r\}$ e k .

Pergunta: existem no máximo k conjuntos cuja união é U ?

Construção da instância de primers

- para cada elemento u_j , crie uma sequência-alvo t_j ;
- para cada conjunto S_i , crie um candidato p_i ;
- declare que p_i cobre t_j exatamente quando $u_j \in S_i$;
- mantenha o orçamento: $k' = k$.

O candidato pode representar um primer ou um par de primers; o modelo recebe a relação de cobertura já calculada pelas restrições experimentais.

O parâmetro é preservado

A construção é polinomial e k' depende somente de k .

A equivalência prova $W[2]$ -dificuldade

Se Set Cover é sim

Se $S_{i_1}, \dots, S_{i_q}$, $q \leq k$, cobrem U , selecione $p_{i_1}, \dots, p_{i_q}$. Todo u_j pertence a algum S_{i_h} ; pela construção, t_j é coberto por p_{i_h} .

Se a cobertura de primers é sim

Se $p_{i_1}, \dots, p_{i_q}$, $q \leq k$, cobrem todos os alvos, então para cada u_j o alvo t_j é coberto por algum p_{i_h} . Pela construção, $u_j \in S_{i_h}$; logo os conjuntos correspondentes cobrem U .

Conclusão

Como Set Cover(k) é $W[2]$ -completo, a OPC é **$W[2]$ -difícil**. Como o modelo é uma instância de Set Cover, também pertence a $W[2]$ e é $W[2]$ -completo.

Problemas parecidos podem cair em lados opostos

problema	parâmetro	resultado
Closest String	distância d	FPT
Closest Substring	número m de strings	W[1]-difícil, mesmo em alfabeto binário

O que muda em Closest Substring?

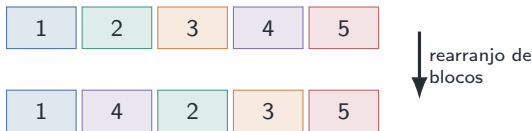
O centro é comparado com **alguma substring** de cada entrada. Além de escolher os símbolos do centro, é preciso escolher onde ele ocorre em cada string.

Ou seja, nem “alfabeto pequeno” nem “poucas strings” garantem FPT. A liberdade combinatória introduzida pelo modelo pode dominar o parâmetro.

Permutações podem representar ordens de genes

Modelo

Cada genoma é representado por uma permutação dos mesmos genes, cada um aparecendo uma vez. Uma métrica mede diferenças por **breakpoints** ou por operações como **swap**, **block-interchange**, **transposição** e **short-block-move**.



Escopo do modelo

Ele abstrai duplicações, perdas e múltiplos cromossomos para isolar a ordem relativa dos genes.

Median e Closest agregam distâncias de modos diferentes

Para entradas $\pi^1, \dots, \pi^m$ e uma métrica d_M :

Median

Encontrar σ que minimize

$$\sum_{i=1}^m d_M(\sigma, \pi^i).$$

Penaliza a soma de todas as discordâncias.

Closest

Encontrar σ que minimize

$$\max_{1 \leq i \leq m} d_M(\sigma, \pi^i).$$

Controla a pior distância.

Parâmetros investigados

Estudamos com parâmetros valor alvo d e o número m de permutações de entrada (Cunha, Sau e Souza, 2024).

O mesmo problema muda muito com o parâmetro

	swap	block-int.	short-block	transposição	breakpoint
Median, d	kernel pol.	kernel pol.	kernel pol.	kernel pol.	kernel pol.
Median, m	para-NP-dif.	para-NP-dif.	aberto	para-NP-dif.	para-NP-dif.
Closest, d	FPT; sem kernel pol.	FPT; sem kernel pol.	FPT; sem kernel pol.	aberto; sem kernel pol.	aberto; sem kernel pol.
Closest, m	para-NP-dif.	para-NP-dif.	aberto	para-NP-dif.	aberto

Como ler “sem kernel pol.”

É um resultado condicional: não existe kernel polinomial em d , a menos que $NP \subseteq coNP/poly$.

Dois contrastes

- d é útil para várias versões; m isoladamente falha mesmo em $m = 3$.
- Um problema pode ser FPT e, ainda assim, não possuir kernel polinomial.

Aproximação troca otimalidade por uma garantia

Seja $OPT(I)$ o valor ótimo e $A(I)$ o valor devolvido por um algoritmo polinomial.

Minimização

A é uma α -aproximação se produz solução viável e

$$A(I) \leq \alpha OPT(I).$$

Maximização

A é uma α -aproximação se produz solução viável e

$$A(I) \geq \frac{OPT(I)}{\alpha}.$$

Garantia de pior caso

Não é apenas “funcionou bem nos testes”: a desigualdade deve valer para toda instância.

Exemplo

Se o ótimo usa 10 primers, uma 2-aproximação garante no máximo 20. Ela pode devolver 10, 11 ou 17, mas nunca mais de 20.

APX e PTAS oferecem níveis diferentes de garantia

classe	garantia	tempo
APX	algum fator constante α	polinomial; α não depende da instância
PTAS	$(1 + \varepsilon)$ para todo $\varepsilon > 0$	polinomial em n para cada ε fixo; o grau pode depender de $1/\varepsilon$
EPTAS	$(1 + \varepsilon)$	$f(1/\varepsilon)n^{O(1)}$
FPTAS	$(1 + \varepsilon)$	polinomial em n e em $1/\varepsilon$

Relações

$$\text{FPTAS} \subseteq \text{EPTAS} \subseteq \text{PTAS} \subseteq \text{APX}.$$

Um problema em APX pode possuir apenas uma aproximação constante distante de 1; um PTAS permite aproximar arbitrariamente o ótimo, pagando no tempo por ε .

O parâmetro agora é a precisão

Em um EPTAS, a explosão associada a $1/\varepsilon$ fica separada do grau de n , exatamente no espírito de FPT.

APX-dificuldade exige reduções que preservam aproximação

Uma redução polinomial de decisão preserva respostas sim/não, mas pode destruir a relação entre os valores das soluções. Para transferir aproximações, usamos reduções com controle quantitativo.

L-redução de A para B

Existem constantes $\alpha, \beta > 0$ e funções polinomiais f, g tais que

$$OPT_B(f(x)) \leq \alpha OPT_A(x)$$

e, para toda solução y de $f(x)$,

$$|OPT_A(x) - val_A(g(y))| \leq \beta |OPT_B(f(x)) - val_B(y)|.$$

Assim, uma solução quase ótima de B volta a uma solução quase ótima de A .

APX-hard e APX-completo

Um problema é **APX-difícil** se todo problema de APX reduz a ele por uma redução que preserva aproximação. Se também pertence a APX, é **APX-completo**. Sob essas reduções, um PTAS para um problema APX-difícil implicaria $P = NP$.

Closest String possui FPT exato e PTAS

Exato e parametrizado

Parâmetro: raio ótimo/permitido d .

Tempo:

$$O^*((d+1)^d).$$

Interessante quando d é pequeno e a resposta precisa ser exata.

Aproximado

Para qualquer $\varepsilon > 0$, há PTAS para Closest String.

Para ε fixo, o esquema enumera uma amostra de tamanho dependente de $1/\varepsilon$, usa-a para restringir as posições relevantes e resolve o restante em tempo polinomial. A melhor solução enumerada tem raio no máximo $(1 + \varepsilon)OPT$.

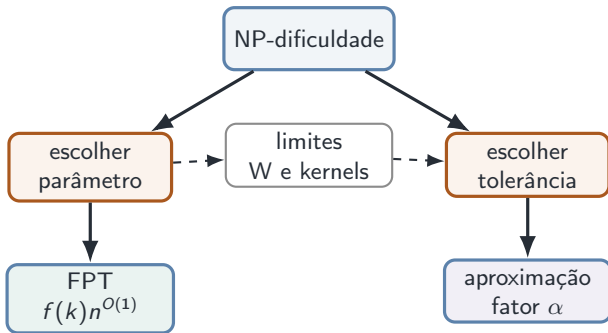
Não há vencedor universal

FPT é indicado quando d é pequeno e exigimos exatidão. O PTAS é indicado quando d pode ser grande e aceitamos erro relativo. Não se espera um EPTAS para o problema, a menos que $FPT = W[1]$.

Parametrização e aproximação podem ser combinadas

estratégia	pergunta	forma típica
FPT exato	a estrutura difícil é pequena?	$f(k)n^{O(1)}$, solução ótima
aproximação	aceitamos perda controlada?	$n^{O(1)}$, fator α
EPTAS	precisão alta é o parâmetro?	$f(1/\varepsilon)n^{O(1)}$
FPT-aproximação	uma aproximação melhora com um parâmetro estrutural?	$f(k)n^{O(1)}$, fator garantido
kernel + aproximação	podemos reduzir e depois aproximar/resolver?	pré-processamento com garantia

Resumo



- **Closest String**: kernelização parcial + árvore de busca limitada;
- **permutações de genes**: parâmetros distintos, kernels, FPT e limites;
- **primers**: cobertura NP-difícil, mas com aproximação logarítmica.

Referências principais

- M. Cygan et al. *Parameterized Algorithms*. Springer, 2015.
- J. Gramm, R. Niedermeier e P. Rossmanith. Fixed-Parameter Algorithms for Closest String and Related Problems. *Algorithmica* 37, 25–42, 2003.
- M. Li, B. Ma e L. Wang. On the Closest String and Substring Problems. *JACM* 49(2), 157–171, 2002.
- M. Cygan et al. Lower Bounds for Approximation Schemes for Closest String. *SWAT*, 2016.
- M. Fellows, J. Gramm e R. Niedermeier. On the Parameterized Intractability of Motif Search Problems. *Combinatorica* 26, 141–167, 2006.
- L. Cunha, I. Sau e U. Souza. On the Parameterized Complexity of the Median and Closest Problems under Some Permutation Metrics. *Algorithms for Molecular Biology* 19:24, 2024.
- L. Bulteau e M. Weller. Parameterized Algorithms in Bioinformatics: An Overview. *Algorithms* 12(12):256, 2019.
- J. Buss e J. Goldsmith. Nondeterminism within P. *SIAM J. Comput.* 22(3), 1993.
- C. Papadimitriou e M. Yannakakis. Optimization, approximation, and complexity classes. *JCSS* 43(3), 1991.
- W. Pearson, G. Robins, D. Wrege e T. Zhang. On the Primer Selection Problem in PCR Experiments. *Discrete Applied Mathematics* 71, 231–246, 1996.
- D. Johnson (1974) e V. Chvátal (1979), análises clássicas do guloso para Set Cover.